# VO IV

## Void Storage Architecture

Typed Storage, Stratified Indexing, Retrieval Runtime, and Integration with the Neurol Substrate

Kevin T.N
Vietnam
keedavinci@gmail.com

March 18, 2026

**Abstract**

This document is the fourth major volume of the Void sequence and carries the name VO IV. If VO I established the philosophy of Void, VO II established the ontology of Void-data, and VO III established the algorithms required to operate on that data, then VO IV defines the architecture required to store, index, execute, and integrate those algorithms in a real system. This volume proposes a typed and stratified Void architecture. It distinguishes storage classes, archive depths, index families, repositories, retrieval and reconstruction services, lifecycle and integrity guards, and the runtime boundaries between Void and the active neurol substrate. The core claim is that a Void system cannot be implemented as a generic blob store or flat vector database. It must be architected as a layered archive of latent, residual, compressed, seeded, traced, contextual, and potential structures.

## Contents

**1 Introduction** **5**
1.1 Why Storage Architecture Comes Last . . . 5
1.2 The Problem to Be Solved . . . 5
1.3 Main Architectural Thesis . . . 5

**2 Reference Position and Continuity** **5**
2.1 Reference Statement . . . 5
2.2 Position in the Sequence . . . 5
2.3 What VO IV Must Preserve . . . 5

**3 Architectural Premises** **5**
3.1 Premise 1: Void Is Not a Trash Bin . . . 5
3.2 Premise 2: Not All Void-Data Is Equally Retrievable . . . 6
3.3 Premise 3: Metadata Is Ontology-Carrying . . . 6
3.4 Premise 4: One Architecture, Many Storage Modes . . . 6

**4 The Global Void Storage Model** **6**
4.1 The Typed Void Store . . . 6
4.2 Stratified by Depth . . . 6
4.3 Meaning of the Model . . . 6

**5 The Canonical Void Object Model** **6**
5.1 Universal Object Envelope . . . 6
5.2 Why an Envelope Is Needed . . . 7
5.3 Payload Polymorphism . . . 7

**6 Class-Specific Storage Families** **7**
6.1 Latent Store . . . 7
6.2 Residual Store . . . 7
6.3 Compressed Store . . . 8
6.4 Seed Store . . . 8
6.5 Trace Store . . . 8
6.6 Archive-Context Store . . . 8
6.7 Potential Store . . . 8

**7 Depth Layers and Their Storage Policies** **9**
7.1 Near-Void . . . 9
7.2 Mid-Void . . . 9
7.3 Deep-Void . . . 9

**8 Index Architecture** **9**
8.1 Why Multiple Indexes Are Required . . . 9
8.2 Canonical Index Families . . . 9
8.3 Class Index . . . 10
8.4 Depth Index . . . 10
8.5 Lineage Index . . . 10
8.6 Semantic Index . . . 10
8.7 Context Index . . . 10
8.8 Activation Index . . . 10

**9 Repository and Storage Interfaces** **10**
9.1 Why Repositories Are Needed . . . 10
9.2 Canonical Repository Set . . . 10
9.3 Shared Repository Contracts . . . 11
9.4 Why a Generic Repository Is Not Enough . . . 11

**10 Storage Layout Options** **11**
10.1 Polyglot Storage Is Acceptable . . . 11
10.2 Why This Is Not Fragmentation . . . 11
10.3 Storage Selection Law . . . 11

**11 Void Runtime Services** **11**
11.1 Need for Runtime Services . . . 11
11.2 Core Runtime Services . . . 12
11.3 Service Topology . . . 12

**12 The Classification Service** **12**
12.1 Responsibility . . . 12
12.2 Inputs and Outputs . . . 12
12.3 Architectural Note . . . 13

**13 The Compression and Seed Services** **13**
13.1 Compression Service . . . 13
13.2 Seed Service . . . 13
13.3 Architectural Relationship . . . 13

**14 The Retrieval and Reconstruction Services** **13**
14.1 Retrieval Service . . . 13
14.2 Reconstruction Service . . . 13
14.3 Why These Must Be Separate . . . 13
14.4 A Canonical Flow . . . 13

**15 The Activation and Re-Entry Service** **13**
15.1 Responsibility . . . 13
15.2 Inputs . . . 14
15.3 Output . . . 14

**16 Integrity and Legality Guards** **14**
16.1 Why Guards Are Architectural, Not Optional . . . 14
16.2 Guard Layer . . . 14
16.3 Failure Modes . . . 14

**17 Lifecycle of a Void Object** **15**
17.1 The Core Lifecycle . . . 15
17.2 Why Lifecycle Matters . . . 15

**18 Drift, Refresh, and Depth Management** **15**
18.1 Need for Lifecycle Maintenance . . . 15
18.2 Maintenance Jobs . . . 15
18.3 Why This Belongs in Storage Architecture . . . 16

**19 Integration with the Neurol Substrate** **16**
19.1 Bridge Boundary . . . 16
19.2 Bridge Flows . . . 16
19.3 Bridge Contracts . . . 16
19.4 Architectural Warning . . . 16

**20 Interaction with Attention, Reward, and Resource** **16**
20.1 Attention . . . 16
20.2 Reward and Resource . . . 16
20.3 Architectural Consequence . . . 17

**21 Sample Source-Level Model** **17**
21.1 Illustrative Object Models . . . 17
21.2 Illustrative Repositories . . . 17
21.3 Illustrative Services . . . 18

**22 Suggested Repository Layout for a Real Codebase** **18**
22.1 Top-Level Layout . . . 18
22.2 Interpretation . . . 19

**23 Testing Architecture** **19**
23.1 Why Testing Is Hard Here . . . 19
23.2 Required Test Families . . . 19
23.3 The Most Important Test . . . 19

**24 Observability and Diagnostics** **19**
24.1 Need for Observability . . . 19
24.2 Recommended Diagnostics . . . 20

**25 Migration, Versioning, and Evolution of the Void Store** **20**
25.1 Why Versioning Matters . . . 20
25.2 Versioning Rule . . . 20
25.3 Migration Philosophy . . . 20

**26 Build Order for Implementing the Void Architecture** **20**
26.1 Recommended Implementation Sequence . . . 20
26.2 Why This Order Works . . . 21

**27 What VO IV Has Established** **21**
27.1 Main Results . . . 21

**28 Conclusion** **21**

# 1. Introduction

## 1.1. Why Storage Architecture Comes Last

Storage architecture appears late in the sequence for a reason. If one begins with storage too early, the architecture silently defines the ontology. That is the wrong order. By letting philosophy, data ontology, and algorithms come first, the present project ensures that the storage system must serve the concepts rather than deform them.

## 1.2. The Problem to Be Solved

VO IV addresses a very specific problem:

*How can a real system preserve Void-data in typed, stratified, retrievable, lineage-respecting, and reconstruction-sensitive form without collapsing it into ordinary storage categories?*

## 1.3. Main Architectural Thesis

The main architectural thesis is the following:

**Design Law 1.1** (Typed stratified storage)**.** *A valid Void implementation must be typed by ontological class and stratified by archival depth. Any architecture that stores all Void-data as one flat undifferentiated mass is conceptually invalid for this series.*

# 2. Reference Position and Continuity

## 2.1. Reference Statement

For the purposes of this series, the root reference is declared as:

**"Nguon: Me Thien Nhien"**

## 2.2. Position in the Sequence

The Void sequence now reads:

1. VO I – Void philosophy;
2. VO II – Void data theory;
3. VO III – Void algorithms;
4. VO IV – Void storage architecture.

## 2.3. What VO IV Must Preserve

VO IV must preserve the class distinctions, depth distinctions, retrieval legality, reconstruction sensitivity, and lineage integrity already established. Its duty is architectural faithfulness.

# 3. Architectural Premises

## 3.1. Premise 1: Void Is Not a Trash Bin

A Void store is not where data goes to disappear. It is where data changes mode of being. Therefore retention policy cannot be modeled as simple deletion fallback.

### 3.2. Premise 2: Not All Void-Data Is Equally Retrievable

Because Void-data has depth and class, storage must preserve retrieval affordances and reconstructive affordances explicitly.

### 3.3. Premise 3: Metadata Is Ontology-Carrying

In ordinary systems, metadata is often merely descriptive. In a Void system, metadata carries ontology. The class, depth, reconstructive cost, lineage, and integrity profile are not optional annotations. They are part of the identity of the object.

### 3.4. Premise 4: One Architecture, Many Storage Modes

Void architecture may use multiple physical storage technologies. The key is not physical uniformity. The key is ontological coherence.

## 4. The Global Void Storage Model

### 4.1. The Typed Void Store

Let the global Void store be denoted by

$$\mathscr{S}_{\mathscr{V}} = \bigcup_{c \in \mathscr{C}} \mathscr{S}_c,$$

where $\mathscr{C}$ is the set of Void-data classes:

$$\mathscr{C} = \{\text{latent}, \text{residual}, \text{compressed}, \text{seed}, \text{trace}, \text{archive_context}, \text{potential}\}.$$

### 4.2. Stratified by Depth

Each class-specific store is further stratified by depth:

$$\mathscr{S}_c = \mathscr{S}_c^{\text{near}} \cup \mathscr{S}_c^{\text{mid}} \cup \mathscr{S}_c^{\text{deep}}.$$

### 4.3. Meaning of the Model

This means that a Void item is stored according to two independent coordinates:

a. what kind of thing it is;

b. how deep into Void it currently lies.

This is the first major structural law of the architecture.

## 5. The Canonical Void Object Model

### 5.1. Universal Object Envelope

Every stored Void item should share a universal envelope, even if internal payloads differ. A canonical schema may be written as:

```
@dataclass
class VoidObject:
    object_id: str
```

```
    class_label: str
    depth_level: str
    activation_potential: float
    reconstruction_cost: float
    lineage_ref: dict
    integrity_profile: dict
    context_bundle: dict
    payload_ref: str
    created_at: int
    updated_at: int
    tags: list[str]
```

### 5.2. Why an Envelope Is Needed

The envelope provides uniform access to the ontological properties of any Void object while allowing the payload itself to remain class-specific.

### 5.3. Payload Polymorphism

The payload behind `payload_ref` may differ radically:

a. a trace packet;

b. a semantic compression bundle;

c. a seed structure;

d. a residual field summary;

e. a contextual archive packet;

f. a potential-space descriptor.

## 6. Class-Specific Storage Families

### 6.1. Latent Store

The latent store holds items close to reactivation. It should optimize for:

a. low reconstruction latency;

b. strong context linkage;

c. active retrieval eligibility.

### 6.2. Residual Store

The residual store preserves partial after-effects and historical path dependence. It should optimize for:

a. compactness;

b. lineage linkage;

c. residual field or attractor retrieval.

### 6.3. Compressed Store

The compressed store holds reduced but reconstructable data. It should optimize for:

a. preservation of reconstructive integrity;

b. type-sensitive decompression paths;

c. compression metadata and versioning.

### 6.4. Seed Store

The seed store holds generative cores. It should optimize for:

a. seed legality checks;

b. generative metadata;

c. lineage and re-entry constraints;

d. low-cost seed activation queries.

### 6.5. Trace Store

The trace store holds event marks and historical signatures. It should optimize for:

a. append-friendly persistence;

b. causal indexing;

c. long-horizon lineage scans.

### 6.6. Archive-Context Store

The archive-context store preserves relational and interpretive conditions. It should optimize for:

a. frame bundling;

b. anchor bundles;

c. contextual retrieval under current semantics.

### 6.7. Potential Store

The potential store is the most speculative. It should optimize for:

a. candidate-space probing;

b. partial descriptors;

c. uncertainty-aware access.

## 7. Depth Layers and Their Storage Policies

### 7.1. Near-Void

Near-Void contains objects recently compressed, backgrounded, or archived but still close to active re-entry. Policies should favor:

a. fast access;

b. strong cache behavior;

c. minimal reconstruction overhead.

### 7.2. Mid-Void

Mid-Void contains objects whose re-entry is possible but requires stronger contextualization or reconstruction. Policies should favor:

a. moderate-cost retrieval;

b. richer lineage and context indexes;

c. slower but still practical reconstruction pathways.

### 7.3. Deep-Void

Deep-Void contains heavily compressed, highly residual, or highly speculative forms. Policies should favor:

a. extreme compactness;

b. durable lineage anchors;

c. explicit uncertainty;

d. delayed or gated reconstruction.

## 8. Index Architecture

### 8.1. Why Multiple Indexes Are Required

A single index is insufficient because Void retrieval may be driven by class, depth, lineage, semantics, context, reconstructive cost, or activation potential.

### 8.2. Canonical Index Families

The architecture should include at least the following index families:

1. class index;
2. depth index;
3. lineage index;
4. semantic index;

5. context index;
6. reconstructive-cost index;
7. latent-activation index;
8. integrity index.

### 8.3. Class Index

The class index partitions by ontological category. It is the first gate for legal retrieval.

### 8.4. Depth Index

The depth index partitions by near, mid, and deep layers. It is crucial for cost-aware routing.

### 8.5. Lineage Index

The lineage index supports ancestry, branch, and origin traversal. This index is essential for lawful seed activation and historical interpretation.

### 8.6. Semantic Index

The semantic index supports similarity and resonance retrieval, especially for compressed and archive-context classes.

### 8.7. Context Index

The context index stores or references anchor bundles, frame bundles, and environmental conditions necessary for contextualized re-entry.

### 8.8. Activation Index

The activation index orders items by latent activation potential under current or scheduled contexts.

## 9. Repository and Storage Interfaces

### 9.1. Why Repositories Are Needed

A Void system should not expose raw storage layout directly to all services. It should use repositories that preserve ontological boundaries.

### 9.2. Canonical Repository Set

A practical repository design might include:

```
class LatentRepository: ...
class ResidualRepository: ...
class CompressedRepository: ...
class SeedRepository: ...
class TraceRepository: ...
class ArchiveContextRepository: ...
class PotentialRepository: ...
```

### 9.3. Shared Repository Contracts

Each repository should implement some shared read interfaces:

a. get by id;

b. query by lineage;

c. query by depth;

d. query by semantic resonance;

e. query by activation potential;

f. store/update with legality checks.

### 9.4. Why a Generic Repository Is Not Enough

A single generic `VoidRepository` without specialized class behavior would obscure class-sensitive legality and reconstruction pathways. It may exist as a facade, but not as the only meaningful level.

## 10. Storage Layout Options

### 10.1. Polyglot Storage Is Acceptable

A proper Void architecture may use multiple physical backends, for example:

a. append-oriented storage for traces;

b. document stores for context bundles;

c. vector-capable stores for semantic retrieval;

d. graph structures for lineage;

e. compressed object stores for deep-seed and deep-archive bundles.

### 10.2. Why This Is Not Fragmentation

This is not architectural fragmentation if the logical ontology remains unified. The substrate is one. The physical persistence may be plural.

### 10.3. Storage Selection Law

**Design Law 10.1** (Backend subordination)**.** *Physical storage backends are selected to serve Void-data classes and operators. Ontological classes must not be redesigned merely to fit a convenient backend.*

## 11. Void Runtime Services

### 11.1. Need for Runtime Services

Storage alone is not enough. The algorithms of VO III require executable runtime services.

### 11.2. Core Runtime Services

At minimum, the architecture should include:

1. classification service;
2. depth management service;
3. compression service;
4. residue extraction service;
5. trace preservation service;
6. seed construction service;
7. retrieval service;
8. reconstruction service;
9. activation and re-entry service;
10. integrity and legality service.

### 11.3. Service Topology

These need not all be top-level network services. In a more native implementation they may become internal recursive modules. What matters is that their responsibilities remain distinct.

## 12. The Classification Service

### 12.1. Responsibility

The classification service takes a candidate non-active or transformed object and assigns it to an ontological class and depth layer.

### 12.2. Inputs and Outputs

Input:

- source object descriptor;
- transformation context;
- lineage metadata;
- contextual metadata.

Output:

- class label;
- depth estimate;
- integrity profile;
- preferred repository target.

### 12.3. Architectural Note

This service sits at the front gate between active substrate and Void storage.

## 13. The Compression and Seed Services

### 13.1. Compression Service

The compression service applies typed compression according to class and future intended use.

### 13.2. Seed Service

The seed service specializes in preserving generative cores and re-entry metadata. It must enforce stronger legality constraints than generic compression.

### 13.3. Architectural Relationship

Compression may produce either compressed data, seed data, trace bundles, or residual bundles depending on policy and class.

## 14. The Retrieval and Reconstruction Services

### 14.1. Retrieval Service

The retrieval service is responsible for finding candidate Void items that are relevant to a query, context, lineage path, or activation signal.

### 14.2. Reconstruction Service

The reconstruction service is responsible for turning retrieved Void items into operationally usable structures where legal and justified.

### 14.3. Why These Must Be Separate

This separation enforces the distinction established in VO II and VO III. Retrieval finds. Reconstruction re-forms.

### 14.4. A Canonical Flow

A canonical flow is:

query → class gate → candidate retrieval → legality check → reconstruction → re-entry decision

## 15. The Activation and Re-Entry Service

### 15.1. Responsibility

This service decides whether a Void item may return to active substrate relevance and in what mode:

a. direct restoration;

b. partial activation;

c. seed expansion;

d. background priming;

e. lineage suggestion only;

f. contextual hint only.

### 15.2. Inputs

Inputs include:

- activation potential;
- current active context;
- reconstruction cost;
- integrity requirements;
- current substrate resource budget.

### 15.3. Output

The output is not always a restored object. It may be a re-entry mode plus a partial or deferred activation plan.

## 16. Integrity and Legality Guards

### 16.1. Why Guards Are Architectural, Not Optional

A Void system can easily become ontologically dishonest if it restores too much from too little. Therefore integrity and legality checks must be built into the architecture rather than delegated to best effort.

### 16.2. Guard Layer

A dedicated guard layer should validate:

a. operator legality for the class;

b. minimum required integrity profile;

c. lineage preservation constraints;

d. contextual fit;

e. resource feasibility of reconstruction.

### 16.3. Failure Modes

If a check fails, the architecture should support:

a. deny retrieval;

b. downgrade retrieval mode;

c. permit trace-only return;

d. require contextual expansion first;

e. queue for delayed re-evaluation.

## 17. Lifecycle of a Void Object

### 17.1. The Core Lifecycle

A canonical lifecycle may be described as:

1. active object leaves foreground relevance;
2. classification service assigns Void class;
3. compression or residue extraction runs;
4. object enters typed repository at suitable depth;
5. indexes are updated;
6. activation potential and drift are maintained over time;
7. retrieval may later occur;
8. reconstruction and re-entry may or may not occur;
9. if re-entered, the active substrate receives it in legal mode.

### 17.2. Why Lifecycle Matters

This lifecycle ensures that Void is not a dead-end. It is a phase space of transformed continuation.

## 18. Drift, Refresh, and Depth Management

### 18.1. Need for Lifecycle Maintenance

Stored Void objects should not be assumed static forever. They may drift deeper, lose reconstructive integrity, gain activation relevance, or become merged into richer bundles.

### 18.2. Maintenance Jobs

Architecturally, the system should run maintenance jobs for:

a. depth promotion;

b. depth demotion;

c. integrity decay tracking;

d. access-based refresh;

e. merge recommendation;

f. stale-potential pruning or freezing.

### 18.3. Why This Belongs in Storage Architecture

Because these policies affect where objects live physically and how they are indexed, they belong partly to architecture, not only to algorithms.

## 19. Integration with the Neurol Substrate

### 19.1. Bridge Boundary

The active neurol substrate and the Void system must meet through explicit bridge boundaries. There should be no hidden leakage between active runtime state and Void repositories.

### 19.2. Bridge Flows

The main bridge flows are:

1. active-to-void compression;
2. active-to-void residue extraction;
3. active-to-void contextual archival;
4. active-to-void lineage summary;
5. void-to-active retrieval request;
6. void-to-active reconstruction return;
7. void-to-active activation suggestion.

### 19.3. Bridge Contracts

Each bridge flow should be formalized as a contract. For example:

```
class ToVoidContract: ...
class VoidRetrievalRequest: ...
class VoidReconstructionResult: ...
class VoidActivationSuggestion: ...
```

### 19.4. Architectural Warning

Void should not be treated as a hidden extension of working memory. That would destroy the distinction between active state and Void state.

## 20. Interaction with Attention, Reward, and Resource

### 20.1. Attention

Attention in the active substrate may decide when Void access is worth the cost. Therefore the storage architecture should expose cheap previews and coarse relevance signals without forcing full reconstruction.

### 20.2. Reward and Resource

Resource and reward systems may also influence Void access. Deep reconstructions are expensive. Some items may be worth re-entry only when strong need or high future utility is expected.

### 20.3. Architectural Consequence

The Void architecture should therefore provide:

a. low-cost metadata scans;

b. activation-potential filters;

c. reconstruction-cost estimates;

d. partial retrieval previews.

## 21. Sample Source-Level Model

### 21.1. Illustrative Object Models

```
from dataclasses import dataclass, field
from typing import Literal, Optional, Dict, Any

VoidClass = Literal[
    "latent", "residual", "compressed", "seed",
    "trace", "archive_context", "potential"
]

DepthLevel = Literal["near", "mid", "deep"]

@dataclass
class IntegrityProfile:
    explicit: float = 0.0
    relational: float = 0.0
    lineage: float = 0.0
    reconstructive: float = 0.0
    semantic: float = 0.0

@dataclass
class VoidObject:
    object_id: str
    class_label: VoidClass
    depth_level: DepthLevel
    activation_potential: float
    reconstruction_cost: float
    lineage_ref: Dict[str, Any] = field(default_factory=dict)
    context_bundle: Dict[str, Any] = field(default_factory=dict)
    integrity: IntegrityProfile = field(default_factory=IntegrityProfile)
    payload_ref: Optional[str] = None
    tags: list[str] = field(default_factory=list)
    created_at: int = 0
    updated_at: int = 0
```

### 21.2. Illustrative Repositories

```
class BaseVoidRepository:
    def get(self, object_id: str): ...
    def put(self, obj: VoidObject): ...
```

```python
    def query_by_depth(self, depth_level: str): ...
    def query_by_lineage(self, lineage_id: str): ...
    def query_by_activation_threshold(self, threshold: float): ...

class SeedRepository(BaseVoidRepository):
    def query_seed_ready(self, threshold: float): ...

class TraceRepository(BaseVoidRepository):
    def query_by_causal_parent(self, parent_id: str): ...
```

### 21.3. Illustrative Services

```python
class VoidClassificationService: ...
class VoidCompressionService: ...
class VoidResidueService: ...
class VoidRetrievalService: ...
class VoidReconstructionService: ...
class VoidActivationService: ...
class VoidIntegrityGuard: ...
```

## 22. Suggested Repository Layout for a Real Codebase

### 22.1. Top-Level Layout

A possible repository section for the Void system might be:

```
src/
  ipod-void/
    input/
    process/
      ipod-classification/
      ipod-depth/
      ipod-compression/
      ipod-residue/
      ipod-trace/
      ipod-seed/
      ipod-retrieval/
      ipod-reconstruction/
      ipod-activation/
      ipod-integrity/
    output/
    display/
    data/
    state/
      void_object.py
      integrity.py
      lineage.py
      depth.py
    contracts/
    tests/

  ipod-void-storage/
    state/
```

```
    repositories/
    indexes/
    maintenance/
    tests/
```

### 22.2. Interpretation

This layout aligns with the recursive architecture already developed in the neurol sequence. The Void system can therefore be implemented as one more native recursive subsystem rather than as an external bolt-on.

## 23. Testing Architecture

### 23.1. Why Testing Is Hard Here

A Void architecture has many more failure modes than ordinary storage systems because it can fail ontologically, not only technically.

### 23.2. Required Test Families

At minimum the architecture should include:

1. class-assignment correctness tests;
2. depth-stratification tests;
3. repository routing tests;
4. retrieval legality tests;
5. reconstruction integrity tests;
6. lineage preservation tests;
7. drift and maintenance tests;
8. neurol-void bridge tests.

### 23.3. The Most Important Test

The single most important family of tests is this: the system must not silently return an object in a stronger ontological mode than the stored class and integrity profile justify.

## 24. Observability and Diagnostics

### 24.1. Need for Observability

Because the Void architecture is deep and class-sensitive, operators need visibility into:

a. how much data lives in each class and depth;

b. which classes are being retrieved;

c. reconstruction cost trends;

d. activation suggestion rates;

e. lineage break incidents;

f. integrity failures.

### 24.2. Recommended Diagnostics

A useful diagnostic suite should expose:

a. class histograms;

b. depth histograms;

c. lineage fan-out maps;

d. retrieval legality failure reports;

e. drift rates by class;

f. seed activation success rates.

## 25. Migration, Versioning, and Evolution of the Void Store

### 25.1. Why Versioning Matters

A Void system that preserves lineage must also preserve schema evolution. Future changes to storage structure should not break the meaning of older objects.

### 25.2. Versioning Rule

Every Void object and every payload schema should carry version markers. Repositories should know how to migrate or adapt older forms under controlled transformation.

### 25.3. Migration Philosophy

Migration should itself be treated as transformation rather than blind overwrite. Older ontological classes should not be flattened merely to satisfy newer storage convenience.

## 26. Build Order for Implementing the Void Architecture

### 26.1. Recommended Implementation Sequence

A sensible build order is:

1. define universal Void object envelope and integrity profile;
2. implement class and depth indexes;
3. implement class-specific repositories;
4. implement classification and routing service;
5. implement compression and trace services;
6. implement retrieval and reconstruction services;

7. implement integrity guard and legality layer;
8. implement drift and maintenance jobs;
9. implement neurol-void bridge contracts;
10. integrate activation suggestions into the active substrate.

### 26.2. Why This Order Works

This order builds from ontology-bearing structures upward toward costly runtime integration.

## 27. What VO IV Has Established

### 27.1. Main Results

By the end of this volume, the following architectural results have been established:

1. A valid Void system requires typed and stratified storage.
2. Class and depth are the first architectural coordinates of Void-data.
3. Metadata in a Void system is ontology-bearing rather than merely descriptive.
4. Multiple indexes are required: class, depth, lineage, semantics, context, cost, activation, and integrity.
5. Repositories must be class-sensitive.
6. Retrieval and reconstruction services must remain separate.
7. Integrity and legality guards are architectural necessities.
8. Void integrates with the active neurol substrate only through explicit bridge boundaries.
9. Drift, maintenance, and versioning are part of architecture, not optional operational details.

## 28. Conclusion

VO IV completes the first full arc of the Void sequence. With this volume, Void now possesses a philosophical foundation, a data ontology, an algorithmic grammar, and a storage architecture. This means the sequence has crossed from speculative language into system design.

The central achievement of this volume is to make explicit that a Void system cannot be implemented as generic archive infrastructure. It must be typed, stratified, guarded, and lineage-aware. It must distinguish retrieval from reconstruction, preserve class and depth as first-class architectural coordinates, and integrate with the active substrate through controlled boundaries rather than hidden memory leakage.

At this point, the Void sequence has become implementable in principle. What remains after this volume is not another conceptual layer of the same order, but the actual source realization of these designs inside a living repository.